

The midterm exam will be on Tuesday, October 8, in the lecture room (IRB 0318) during our regular class time (2:00–3:15 pm). You may consult textbooks and notes during the exam, but you may not use any other resources (in particular, electronic devices are not allowed).

The following problems may help you prepare for the midterm. They are not to be handed in. Solutions are not available, but of course you are welcome to discuss these problems on Piazza and during office hours.

Disclaimer: Many of these problems are taken from exams in previous offerings of CMSC 451 when the emphasis of the course may have been different. These problems may not precisely reflect the difficulty level or distribution of material to be covered on the exam.

1. *Short answer questions.*

For the following questions, explanations are not required, but can be given for partial credit.

- (a) Recall that in the Gale-Shapley algorithm as discussed in class, companies make offers to students. True or false: once a *company* makes an offer that is accepted, that company is guaranteed to remain engaged (to some student) for the remainder of the algorithm.
- (b) Recall that in the Gale-Shapley algorithm as discussed in class, companies make offers to students. Which of the following describes how the sequence of students engaged to a given company changes over time? The company's preference for the student to whom it is engaged...
 - i. ... strictly increases in each iteration.
 - ii. ... may either increase or stay the same from one iteration to the next.
 - iii. ... may either decrease or stay the same from one iteration to the next.
 - iv. ... strictly decreases in each iteration.
- (c) Suppose that in an instance of the stable matching problem, student s is the top choice of company c and company c is the top choice of student s . True or false: in any stable matching, c and s must be paired with each other.
- (d) Consider the following greedy algorithm for the stable matching problem (as formulated in class): go through all companies in an arbitrary order and assign each to its highest-preference available student. Give a minimal counterexample showing that this algorithm does not necessarily produce a stable matching. Explain why the matching in your example is unstable.
- (e) Consider running the Gale-Shapley algorithm in which n companies make offers to n students. What is the running time in each of the following scenarios? Express your answer with big- Θ notation.
 - All companies have the same preferences.
 - Each company's top choice is a different student.

- (f) What is the asymptotic running time of the following program as a function of n ? (Express your running time using Θ notation.)

```
f(n):
    set i = n
    print "the output goes on"
    while i > 0
        for j = 1 to i
            print " and on"
        endfor
        set i = i/2
    endwhile
```

- (g) Consider the following algorithm that takes as input the adjacency list representation of an n -vertex undirected graph G .

```
ExploreTriangles(G):
    for every vertex v of G
        for every neighbor u of v
            for every neighbor w of u
                if w is a neighbor v then print "found a triangle!"
            endfor
        endfor
    endfor
```

- i. Suppose every vertex of G has degree at most 100. What is the running time of this algorithm as a function of n ? Express your answer using Θ notation.
 - ii. Suppose every vertex of G has degree $\Theta(\sqrt{n})$. What is the running time of this algorithm as a function of n ? Express your answer using Θ notation.
- (h) For each of the following functions, indicate whether $f(n)$ is in o , ω , or Θ of $g(n)$. In this problem, $\log$ denotes the base-2 logarithm.
- i. $f(n) = 10^{n/2}$, $g(n) = 5^n$
 - ii. $f(n) = n^2 + 10^{100} n \log n$, $g(n) = 10^{100} n^2 + n \log n$
 - iii. $f(n) = \log \sqrt{n}$, $g(n) = \sqrt{\log n}$
- (i) List the following functions in increasing asymptotic order. If two functions are asymptotically equivalent, then indicate this.

$$(i) \ n^{3/2} + n^{2/3} \quad (ii) \ n(\log n)^2 \quad (iii) \ 4^{\log n} \quad (iv) \ \max\{2000n^2, n^3\}$$

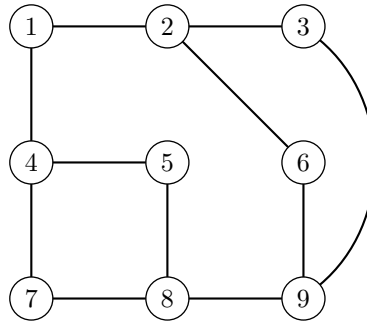
(In this problem, $\log$ represents the base-2 logarithm.)

- (j) For each of the following functions, indicate whether $f(n)$ is in o , ω , or Θ of $g(n)$. In this problem, $\log$ denotes the base-2 logarithm.
- i. $f(n) = 27n^3$, $g(n) = \sum_{i=1}^n \sum_{j=1}^i (3j + 5)$
 - ii. $f(n) = n \log \log n$, $g(n) = (\log n)^2$
 - iii. $f(n) = 2^n$, $g(n) = 5^{n/2}$

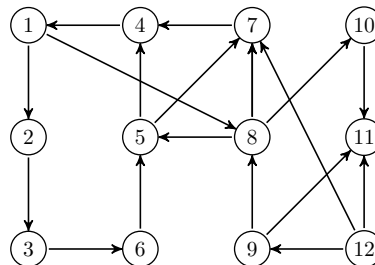
- (k) For each of the following functions, indicate whether $f(n)$ is in o , ω , or Θ of $g(n)$.
- $f(n) = \sum_{i=1}^n (451i^2 + 2000)$, $g(n) = n^3$
 - $f(n) = 3^{2n}$, $g(n) = 2^{3n}$
 - $f(n) = n + \sqrt{n} \log_2 n$, $g(n) = n \log_2 n$
- (l) List the following functions in increasing asymptotic order. If two functions are asymptotically equivalent (i.e., related by big- Θ), then indicate this.

(i) $\sqrt{n/2000}$ (ii) $2000 \log_2 n$ (iii) $n^{1/2000}$ (iv) $2^{\frac{1}{2} \log_2 (n-2000)}$

- (m) What is the maximum number of edges in an undirected graph with n vertices, in which each vertex has degree at most k ?
- (n) How many edges are there in an n -vertex forest with c components?
- (o) How many edges are there in a graph with $4n$ vertices, where each vertex has degree $n + 7$? Give both an exact expression and an asymptotic expression of the form $\Theta(n^c)$ for some constant c .
- (p) Give an example of a graph for which the breadth-first search algorithm (as presented in class) can output different BFS trees starting from some particular initial vertex, depending on the order in which the neighbors of vertices are considered. Your example should involve as few vertices as possible.
- (q) Is the following graph bipartite? Justify your answer by drawing a search tree produced by running an algorithm discussed in class, and indicate what property of this tree illustrates the bipartiteness or nonbipartiteness of the graph.



- (r) Is the following digraph strongly connected? Explain why or why not.



- (s) True or false: Every directed acyclic graph has a unique topological ordering. If true, explain why; if false, give a minimal counterexample.

- (t) Suppose G_1 is a directed graph that is *not* a DAG (directed acyclic graph). Let G_2 be a directed graph obtained by reversing the directions of all edges of G_1 . Which of the following can you infer from this information alone?
- G_2 has a unique topological ordering
 - G_2 has one or more topological orderings
 - G_2 may or may not have a topological ordering
 - G_2 does not have a topological ordering
- (u) Consider the following algorithm that takes as input a directed graph G and outputs a list of vertices of G :

TS(G):

```

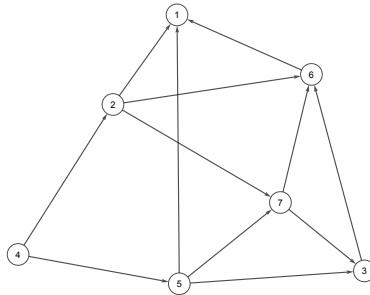
    if  $G$  is empty then
        return an empty list
    else
        let  $v$  be a vertex of  $G$  with outdegree 0
        return a list with  $v$  appended to the end of TS( $G \setminus v$ )
    endif

```

Here $G \setminus v$ denotes the graph G with vertex v and all associated edges removed.

True or false: If G is a DAG, then TS(G) outputs a topological ordering of G .

- (v) Give a topological ordering of the following DAG:



- (w) A *maximum* spanning tree of a weighted, connected graph is a spanning tree with the largest possible total weight of its edges. True or False: If we modify Prim's algorithm to select the non-tree vertex with the *highest* attachment cost, it will produce a maximum spanning tree.
- (x) Recall that in the *interval scheduling problem*, we are given a list of start times s_i and finish times f_i for all $i \in \{1, 2, \dots, n\}$, and our goal is to find a largest possible subset of the intervals $[s_i, f_i]$ such that no two intervals overlap. Consider the following greedy algorithm for interval scheduling:

```

    Sort the intervals by starting times
    While there are any remaining intervals
        Add an interval with the latest starting time to the schedule
        Delete all conflicting intervals
    Endwhile

```

True or false: This algorithm outputs an optimal schedule.

2. *Degree sums in undirected graphs.*

Suppose $G = (V, E)$ is an undirected graph given in adjacency list format. Recall that this format provides a linked list of the neighbors of each vertex (you should assume that the format does not directly provide the degree of a vertex). For any vertex v , let $\text{sumdegree}[v]$ denote the sum of the degrees of the neighbors of v : e.g., if v has neighbors w_1, w_2 , and w_3 with respective degrees 3, 6, 2, then $\text{sumdegree}[v] = 3 + 6 + 2 = 11$. Show how to construct the array of all $\text{sumdegree}[v]$ values (one value for each $v \in V$), in $O(n + m)$ time. (As usual, we let $n = |V|$ and $m = |E|$.) Explain why your algorithm has the desired running time and why it is correct.

3. *Strong connectivity.*

Let $n \geq 2$.

- (a) Prove that any *strongly-connected* n -vertex directed graph G has at least n edges. (Hint: Start your proof by making a useful observation about the out-degree of each vertex.)
- (b) For each $n \geq 2$, give an example of a strongly-connected directed graph G with n vertices and exactly n edges.

4. *Computing girth.*

- (a) Suppose you are given as input an undirected graph $G = (V, E)$ in adjacency list format. Describe an efficient algorithm that, for a given edge $\{u, v\} \in E$, computes the length of the shortest cycle in G that includes this edge. Prove that your algorithm is correct and analyze its running time as a function of $|V|$ and $|E|$.
- (b) The *girth* of a graph G is the length of a shortest cycle in G . Give an efficient algorithm that computes the girth of a graph given in adjacency list format. Prove that your algorithm is correct and analyze its running time as a function of $|V|$ and $|E|$. (Hint: use the algorithm from the previous part as a subroutine.)

5. *Teammates.*

You are given a list of n players and would like to assign each player to exactly one team. The two teams do not need to have the same size, and one of them could even be empty. Each player provides a list of other players that they like, and you can only assign two players to the same team if both of them like each other.

Design an efficient algorithm to find a valid assignment of the players to teams, or output “impossible” if no valid assignment exists. Prove that your algorithm is correct, and analyze its asymptotic running time as a function of the input size, giving the tightest possible bound.

6. *Recognizing DAGs.*

Describe an efficient algorithm that, given a directed graph G in adjacency list representation with n vertices and m edges, determines whether G is a directed acyclic graph (DAG). Prove that your algorithm is correct and analyze its running time as a function of n and m . For full credit, your algorithm should run in time $O(m + n)$.

7. *Shortest path and minimum spanning trees under a uniform shift.*

Let G be a weighted undirected graph in which all edge weights are distinct. Construct a new graph G' by adding a fixed positive constant k to the weight of every edge in G .

- (a) Prove or disprove: The shortest path tree in G rooted at a vertex s of G (i.e., the tree produced when Dijkstra's algorithm is run on G starting from s) is the same as the shortest path tree in G' rooted at s .
- (b) Prove or disprove: The minimum spanning tree of G is the same as the minimum spanning tree of G' .

8. *Maximum-cost path in a weighted DAG.*

You are given a directed acyclic graph $G = (V, E)$. Each edge $(u, v) \in E$ of this graph is associated with a weight $w(u, v)$, which can be any positive real number. The cost of a path is defined to be the sum of the weights of the edges along the path. Present an efficient algorithm that computes the maximum cost of any path in G . Justify your algorithm's correctness and establish its running time.

9. *Minimum spanning tree of a near-tree.*

An n -vertex graph is a *near-tree* if it is connected and has n edges. Give an algorithm with running time $O(n)$ for finding a minimum spanning tree of a weighted near-tree. Prove that your algorithm is correct and has the desired running time.

10. *Borůvka's algorithm.*

In 1926, Otakar Borůvka published a method for constructing an efficient electrical network for Moravia. This minimum spanning tree algorithm predates both the Kruskal and Prim algorithms. Given a connected, undirected input graph G with positive edge weights w such that no two edges have the same weight, it works as follows:

Borůvka(G, w):

Let F be a forest with the same vertices as G , but no edges

While F has more than one component

Let S be an empty set

For each component C of F

Let $\{u, v\}$ be the lowest-weight edge with u in C and v not in C

Add edge $\{u, v\}$ to S

Endfor

Add the edges in S to the forest F

Endwhile

In both parts of this problem, you should assume that G is a connected, undirected graph and that no two edges of G have the same weight.

- (a) Prove that Borůvka's algorithm outputs a minimum spanning tree of G . (You are *not* asked to analyze its running time.)
- (b) True or false: Borůvka's algorithm and Prim's algorithm always output the same minimum spanning tree.

11. *Making change.*

Describe a greedy algorithm for making change using quarters, dimes, nickels, and pennies. Assume that the input is given as the number of cents. Prove that your algorithm yields the minimum number of coins. (Hint: For partial credit, prove this for the simpler binary sequence of denominations: 1, 2, 4, 8, 16.)

12. *Dominating sets.*

Given a graph $G = (V, E)$, a vertex subset $V' \subseteq V$ is called a *dominating set* if every vertex of G is either in the set V' or is a neighbor of a vertex in V' . In the *dominating set problem* you are given a graph G and the objective is to compute a dominating set of smallest size.

- (a) Describe a greedy algorithm for computing a small dominating set. Your algorithm should run in time that is polynomial in $n = |V|$.
- (b) Present an example to show that your greedy algorithm is not optimal.

13. *Making pizza.*

You own a pizzeria where customers place orders for n different pizzas $P_1, \dots, P_n$. Each pizza P_i has an associated *preparation time* $p_i \geq 0$ and *baking time* $b_i \geq 0$. You can only prepare one pizza at a time, but your oven can bake any number of pizzas simultaneously. (Of course, each pizza must be prepared before it is baked.) The *finishing time* is the time the last pizza is done.

(*Example:* Say $n = 2$ with $(p_1, b_1) = (3, 6)$ and $(p_2, b_2) = (4, 1)$. Then preparing P_1 followed by P_2 means that you finish preparing P_1 at time $0 + p_1 = 3$, so it finishes baking at time $3 + b_1 = 9$; and you finish preparing P_2 at time $3 + p_2 = 7$, so it finishes baking at time $7 + b_2 = 8$. The finishing time in this case is thus 9.)

Describe an efficient greedy algorithm that finds an order for preparing all the pizzas that minimizes the finishing time. Analyze the running time of your algorithm, and prove that it finds an order with the optimal finishing time. (Hint: Consider an exchange argument.)

14. *Scheduling to minimize lateness revisited.*

Recall that in the problem of scheduling to minimize lateness, we are given n jobs with durations t_i and deadlines d_i , and our goal is to schedule the jobs (i.e., choose their nonnegative starting times) such that no two overlap, minimizing the lateness (the longest time after its deadline that it takes for some job to finish). Let

$$\tau_j = \sum_{i: d_i \leq d_j} t_i$$

denote the sum of the durations of all jobs with deadlines no later than the deadline of job j .

- (a) Prove that the lateness of any feasible schedule (i.e., any schedule in which no two jobs overlap) is at least $\tau_j - d_j$ (for any $j \in \{1, \dots, n\}$).
- (b) Recall that the greedy algorithm we discussed in class simply schedules the jobs in order of increasing deadline. Use the previous part to prove that this is optimal. (You cannot appeal to the proof from class; you have to use the above fact to give a new proof.)

15. *Greedy interval scheduling.*

Recall the following problem, called the Interval Scheduling Problem. We are given a set of n activity requests, each of which has a specified start time s_i and finish time f_i , corresponding to the interval $[s_i, f_i]$. The objective is to compute the maximum number of activities whose corresponding intervals do not overlap.

- (a) In the Earliest Activity First (EAF) greedy algorithm, we schedule the activity with the earliest start time, remove all activities that overlap it, and repeat until no more activities remain. Give an example to show that EAF is not optimal.

- (b) In the Shortest Activity First (SAF) greedy algorithm, we schedule the activity with the smallest duration ($f_i - s_i$), remove all activities that overlap it, and repeat until no more activities remain. Give an example to show that SAF is not optimal.

16. *Road trip.*

Suppose you are going on a road trip with a car that can travel for 300 miles on a full charge. There are charging stations at distances $0, x_1, x_2, \dots, x_n$ along the way, where $0 < x_1 < x_2 < \dots < x_n$. Present an efficient algorithm that determines the fewest number of charging stations you must stop at to reach the station at distance x_n without running out of power along the way, or outputs “not possible” if it is impossible to reach the last station. Prove that your algorithm is correct and analyze its running time as a function of n . (Hint: Try a greedy strategy.)

17. *Air mail.*

Suppose you need to send m identical packages to the same location. There are n different airplanes available for shipments. For $i \in \{1, 2, \dots, n\}$, plane i has capacity p_i (i.e., it can carry at most p_i packages). The cost of sending plane i fully loaded is c_i . If you only use some of the plane’s capacity, then you only pay for the fraction that you use (e.g., if you put $p_i/2$ packages on plane i , so it is only half full, then you pay $c_i/2$). You may assume that $\sum_{i=1}^n p_i \geq m$, i.e., the planes have enough total capacity to carry all the packages.

Describe an efficient greedy algorithm that, given the number of packages m , the plane capacities p_i , and the plane costs c_i , determines how to send the packages at the lowest possible cost. Prove that your algorithm is correct and analyze its asymptotic running time as a function of n .

18. *Maximum ordered ratio.*

Suppose you are given as input a sequence of numbers $a_1, a_2, \dots, a_n$ with $n \geq 2$. Your goal is to find the largest ratio between two of these numbers where the numerator occurs after the denominator in the sequence. In other words, you would like to compute

$$\max \left\{ \frac{a_i}{a_j} : i, j \in \{1, 2, \dots, n\} \text{ with } i > j \right\}.$$

Describe a divide-and-conquer algorithm to solve this problem. Prove its correctness and analyze its running time. For full credit, your algorithm should run in time at most $O(n \log n)$.

19. *Buy low, sell high.*

You are given as input a list $p_1, \dots, p_n$ of daily share prices of a certain stock for n consecutive days. Your goal is to determine the highest possible profit you can make by buying one share of the stock on some day of your choice and selling it on some later day of your choice, where you are required to hold the stock for at least one additional day before selling it (so for example, if you choose to buy it on day 7 at price p_7 , you cannot sell it on day 8, but you could sell it on day 9 at price p_9 , or on some later day).

Give an algorithm that solves this problem as efficiently as possible. Prove that your algorithm is correct and analyze its running time. For full credit, your algorithm should run in time $O(n \log n)$. (Hint: Use divide and conquer.)

20. *Find the missing number.*

You are given an unsorted array A of length $n - 1$ that contains each integer from $\{1, \dots, n\}$ except for one value that is missing. (Assume n is a power of 2 for simplicity.) Your goal is to

design a divide-and-conquer algorithm that determines the missing value. Note that the trivial algorithm that first sorts the array uses $O(n \log n)$ comparisons.

- (a) Describe a divide-and-conquer algorithm that solves this problem, and briefly argue why your algorithm is correct. (*Note:* There are many algorithms for this problem, but this question is specifically asking for a divide-and-conquer algorithm. For full credit, the running time of your algorithm should be asymptotically better than the trivial algorithm.)
- (b) Let $T(n)$ denote the running time of your algorithm as a function of n . Write down a recurrence for $T(n)$.
- (c) Solve the recurrence from the previous part, and express $T(n)$ using Θ notation. Explain how you arrived at your solution.

21. *Counting mixed-parity inversions.*

Given a list $A = (a_1, \dots, a_n)$ of integers, a *mixed-parity inversion* is a pair a_i and a_j such that $i < j$, $a_i > a_j$, and a_i and a_j are of different parities. (In other words, it is an inversion in which one number is even and the other is odd.)

Design an $O(n \log n)$ -time algorithm that counts the number of mixed-parity inversions in a list A containing n elements. Justify your algorithm's correctness and derive its running time.

22. *Counting vowels before consonants.*

Suppose you are given a string s of length n whose symbols are either vowels (A, E, I, O, or U) or consonants (all other letters). You would like to determine the number of times a vowel appears before a consonant, i.e., the number of ordered pairs $(i, j) \in \{1, 2, \dots, n\}^2$ with $i < j$ such that s_i is a vowel and s_j is a constant. For example, in the string IRIBE there are three such pairs, $(1, 2)$, $(1, 4)$, $(3, 4)$.

Design a divide-and-conquer algorithm that solves this problem in time $O(n \log n)$. Prove that your algorithm is correct and show that it has this running time. (Note that it is possible to solve the problem even faster, but you are specifically asked to give a divide-and-conquer algorithm, which should begin by dividing the list in half and solving the original problem on the left and right halves.)

23. *Increasing triples.*

Given a list of n integers $x = (x_1, x_2, \dots, x_n)$, an *increasing triple* is a triple of indices $i, j, k \in \{1, 2, \dots, n\}$ with $i < j < k$ and $x_i < x_j < x_k$. For example, the input $(8, 6, 7, 5, 3, 0, 9)$ contains an increasing triple at indices $(2, 3, 7)$. Design a divide-and-conquer algorithm that determines whether x contains an increasing triple in time $O(n \log n)$. Prove that your algorithm is correct and that it has the desired running time. (Hint: You should be able to determine whether a string x has an increasing *pair*—i.e., a pair of indices i, j with $i < j$ and $x_i < x_j$ —in time $O(n)$.)